

TRAIL OF BITS — PATCH THE PLANET

Auditing Scapy With Agents

How eleven agent runs became 129 reports and fixes the maintainers were willing to read.

PATCH THE PLANET

The Initiative

Patch The Planet is an OpenAI-funded initiative run by Trail of Bits to improve the security of widely used open-source software.

Trail of Bits engineers conduct focused, human-led security audits using frontier cyber models to accelerate vulnerability discovery, then validate findings, develop fixes, and work with maintainers to get those fixes upstream.

Scapy is one of the selected FOSS projects.

FUNDED BY

OpenAI

RUN BY

Trail of Bits

THIS TALK

Scapy

2026-08-24 → 2026-08-28

The Engagement

11

search runs, all against the
same version of the code

36.0

hours of agent time

670

separate agent sessions

1

human auditor, five days

We did not fuzz. OSS-Fuzz has been throwing random input at the obvious parts since 2020, so we looked where it never reaches: `scapy/contrib/**`, code that reassembles streams, and the parts of Scapy that reply to traffic automatically.

RESULTS — AS OF 2026-09-14

The Funnel

471

crashes and bugs reported by
the runs

265

real, separate bugs once we
removed duplicates

129

sent to the maintainers: 92
private reports, 37 pull requests

80

our patch was merged
(as of 2026-09-14)

85% of our patches were merged, while 89% of identified issues were resolved. The rest of this talk is about how we got from 265 bugs to 129 things worth a maintainer's time.

THE PROBLEM THIS PROCESS SOLVES

An agent pointed at a parser will hand you a hundred crashes in an afternoon. Plenty of them look serious. Only some of them are.

Filing the wrong ones costs the maintainer their time, and it permanently costs you their willingness to review the right ones. Our approach is not about finding more bugs. It is about making each bug prove itself before a person looks at it, and making the machine do the work of ruling things out.

THE APPROACH

What We Do Differently

01 Start with their scope

It is the cheapest check. Everything that fails the project's own definition of a bug is recorded, but not reported.

02 Prove it with a script

A script runs the code and records what actually came back. The agent does not get to tell us the result it hoped for.

03 Adversarial review

A fresh agent, with no memory of the first, is told to disprove the finding. Two agents agreeing proves little if both are trying to agree.

04 Write it all down, review it after

Looking across all findings reveals the one fix upstream of twenty symptoms and allows us to chain unrelated findings.

05 Exhaustive testing

Applying all patches to one tree and running the full test suite is the only way to catch a fix that breaks a dependency. Measuring the cost of a patch steers the agents towards writing better code.

06 Ask the maintainers what good looks like

Style, size, and shape of a fix are their call. We wrote their answers down and held every later fix to them.

Every check exists because something went wrong on an earlier audit. None of this is theoretical.

STEP 01 – START WITH THEIR SCOPE

Their Definition Of A Bug, Not Ours

What the project treats as a security bug is the most important guardrail you give the agents.

SCAPY/PACKET.PY:1139-1151

```
try:
    p = cls(s, _internal=1, ...)
except Exception:
    if conf.debug_dissector:
        raise
    p = conf.raw_layer(s)
```

With `conf.debug_dissector = 0` (the default) any exception during dissection becomes a Raw layer.

Scapy policy dictates **this is not a vulnerability**.

- 1 Does the error reach the program using Scapy, instead of being swallowed to Raw?
- 2 Could the packet actually be sent over a real network?
- 3 Does it arrive from outside, rather than being built by hand locally?
- 4 Is `conf.debug_dissector = 0`?
- 5 Does the exploit work on a default configuration?

Candidates must answer these to be reportable on their own.

STEP 02 – PROVE IT WITH A SCRIPT

A Report Is A Claim. A Receipt Is Evidence.

Show that it happens (Positive)

The failure occurs on the exact tested version, starting from raw bytes.

Show what does not happen (Negative)

An identical packet, missing only the trigger, is handled fine. Without that, you have not shown the trigger caused it.

Show that it reaches the user (Escape)

The error comes out of sniff or rdpicap instead of being swallowed. A script checks this; the agent cannot just claim it.

Show the setup (Environment)

Record exact versions and configurations

```
git -C upstream checkout -- .
run_repro.py FIND-NNN --phase vulnerable
# the bug lands

git -C upstream apply patches/FIND-NNN.patch
run_repro.py FIND-NNN --phase fixed
# same script, same input, now it doesn't
```

Both sets of output ship. The maintainer sees the bug happen and sees it stop, measured identically, instead of taking our word for it.

41	41	41 shown to reach	41
bugs	reproductions	the caller	fixes

Told To Disprove It

Inside the run

Agents were sent to actively attack their own run's candidate bugs. 129 checking jobs were executed across the runs, instructed to reproduce the bug or knock it down.

Outside the run

One fresh reviewer per bug, with zero memory of the prior work. They verify: Does the failure happen as described? From raw bytes? On default settings? Escaping the Raw fallback?

Cite the receipt

Every claim cites the file and line that backs it, and a set share of those citations gets checked by someone else rather than whatever anyone has attention for.

We turned this mechanism on ourselves, telling an agent to attack our own summary of the runs. It found seven real mistakes in it.

Every mistake was a judgement call gone wrong at the edges.

A blank field counted as “unknown” when it meant “no”. A search pattern quietly skipped six entries. Something matched on a filename when it should have matched the full path.

Nothing was a misreading of clear data. The borderline calls that were wrong, and attacking our own work is what found them.

STEP 04 - WRITE IT ALL DOWN, REVIEW IT AFTER

Twenty-Six Bugs, One Guard

Twenty-six Scapy layers could each be made to stop a capture with one short packet — twenty-six files, written by different people over a decade.

DCE/RPC · LDP · Diameter · TLS · STUN · MQTT · RTSP ·
SSH · BGP · DNS-over-TCP · SOME/IP · DoIP · PostgreSQL
· Modbus · MySQL · DICOM · TACACS+ · OpenFlow · ...

Twenty-six reports would have been correct, noisy, and annoying to review. Our patch sat upstream of all of them because we *write it all down and review after*.

One place in `sessions.py` forwards packets without catching errors.

1

advisory

~20

lines in
`sessions.py`

26→0

broken before,
broken after

A Test That Cannot Fail

Delete the guard

Delete the check the fix adds, confirm the new regression test fails, then put the check back. On an earlier audit we shipped four tests that could not fail. This is the step that catches that.

Run their CI, not ours

Run the project's own test commands, **with every fix applied together**. A failing check on their side is a rejection, regardless of whether our fix was correct in isolation.

Reproduce it yourself

Many of the bugs we identified take one line to reproduce. One person runs every reproduction by hand instead of relying on an agent's account.

Report a claim as a claim. A report that says “verified” when it means “read the code” is the most expensive kind of error, because everything downstream inherits it.

STEP 05, CONTINUED – AND MEASURE WHAT EACH FIX COSTS

What The Fix Costs

+4.8%

worst slowdown, across 41 tests

0

over the 10% limit that forces
a rewrite

20/41

too small to measure on the
test machine

15/41

made normal traffic faster — the
best by 44%

An agent forced to publish a cost number stops reaching for lazy solutions (extra passes over data, re-walking structures). Fifteen fixes running faster than the code they replaced is proof of this steering effect.

“No measurable difference, here are the numbers” is an answer a maintainer can check. “LGTM” is not.

The Maintainers Set The Bar

The best results came from asking the maintainers what a good fix looks like, writing it down, and holding every later fix to it. Scapy's maintainers gave this audit hours of their own time, day after day, for a week. None of them do this for a living.

One representation throughout, rather than converting.

Simple shape over clever shape (even sacrificing reach).

Conformance fixes must never reject valid traffic.

Minimal diffs. No refactoring adjacent code. Avoid new imports, files, or methods.

New tests go in the existing suite file.

One line of comment, not a paragraph.

"We don't need the massive comment, otherwise OK."

Checking our other PRs found ten more long comment blocks. The problem was specific: every one was preemptively arguing against an objection we expected a reviewer to raise.

If a comment argues rather than explains, it belongs in the pull request.

CLOSING

What The Process Cannot Do

Define Good Code

It cannot tell you what a project's owners consider good code. That only arrives as feedback over time.

Stop Local Tidiness

Each patch adds a helper that looks correct on its own. The maintainer is forced to read the sum of all.

Decide Scope

Not everything that looks like a bug is one. Flexibility is the point of a protocol analyzer. Scope is the maintainer's call.

The machine can make a finding prove itself. It cannot own the codebase, and it should not try to.